Class 19: Lab — Streaming Pasteurization Sensors

Data-driven Systems Engineering

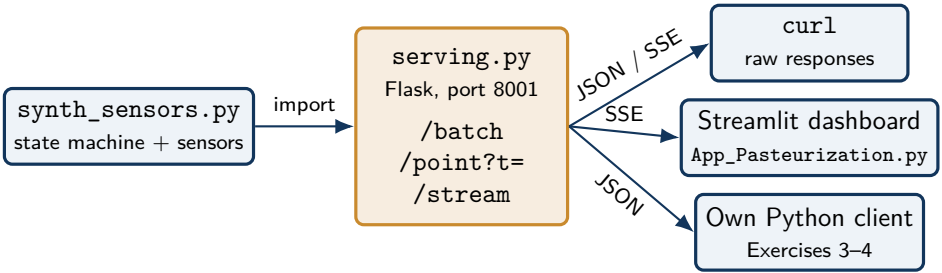
Agenda

- Goal of the lab: from synthetic sensor generation to a live stream that can be consumed.
- `synth_sensors.py`: states, sensors, and the derived feature `dTdt`.
- `serving.py`: the batch, point, and streaming endpoints.
- Guided exercises: run, inspect, request, stream, and detect drift.
- What to observe, and common pitfalls.

Lab overview and setup

1. From the repository root, install the dependencies: `pip install -r requirements.txt`.
2. Two files are used: `pasteurization/synth_sensors.py` and `pasteurization/serving.py`.
3. `synth_sensors.py` does not depend on Flask; it can be run and inspected on its own.
4. `serving.py` imports the simulation functions from `synth_sensors.py` and exposes them over the Hypertext Transfer Protocol (HTTP).

Architecture of the lab



- `/batch` returns one complete simulated batch as a JSON array; `/point?t=` returns a single reading; `/stream` replays one batch as Server-Sent Events (SSE), one event per second.
- The generator has no knowledge of HTTP, and the clients have no knowledge of the simulation: each layer can be tested in isolation.

Sensors and states in the code

```
# Sensors (7): Temperature, pH, Kappa, Mu, Tau, Q_in, Q_out, P
# Derived: dT/dt

PROD_RANGES = {
    "Idle":       (60, 120),
    "Fill":       (30, 60),
    "HeatUp":     (30, 45),
    "Hold":       (15, 25),
    "Cool":       (30, 60),
    "Discharge": (60, 120),
}
```

- From `pasteurization/synth_sensors.py`: the docstring counts seven sensors, but inflow and outflow are separate channels, so each sample is $\mathbf{x}_t \in \mathbb{R}^8$, plus the derived `dTdt`.
- Six production states, each with a duration drawn uniformly from the given range (in seconds) for every batch.

The batch lifecycle as a state machine

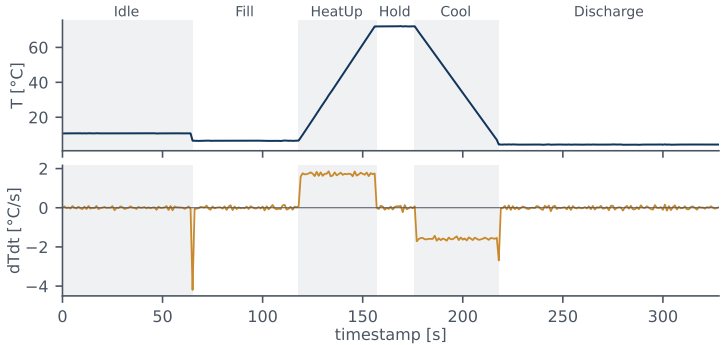


- Each function `seg_*` in `synth_sensors.py` generates the signals of exactly one state.
- The sequence of states is fixed; only the durations are random. The state is therefore a deterministic function of the time elapsed since the start of the batch, given the sampled durations.

Exercise 1: generate and inspect a batch

1. Run `python synth_sensors.py` from the directory `pasteurization/`; it simulates 1,000 batches (about 324,000 samples, a few seconds).
2. Open `synthetic_pasteurization_signals.csv` and verify that the column `state` contains exactly the six states, in the expected order within each `batch_id`.
3. Plot `T` and `dTdt` for a single `batch_id` and identify the phases HeatUp and Cool.
4. Verify that `dTdt` changes sign where the temperature trend reverses.

Exercise 1: expected result



- One batch: the temperature rises during HeatUp, stays at the set point of 72 °C during Hold, and falls during Cool.
- `dTdt` is approximately constant and positive during HeatUp, negative during Cool, and close to zero elsewhere; isolated spikes occur where a state begins with a jump in temperature (e.g. at the start of Fill).

The derived feature in the code

```
def simulate_batch(bid, start_time=0.0):
    ...
    frames = simulate_production_cycle(pdur, ctx)
    df, t = to_df(frames, start_time, bid)
    df["dTdt"] = df["T"].diff().fillna(0.0) / DT
    return df, t
```

- `dTdt` is the backward finite difference $(T_t - T_{t-1})/\Delta t$, the rate-of-change feature constructed in Class 15.
- It is causal: it uses only the current and the previous sample, so it can also be computed online on the stream.

Exercise 2: serve the simulation over HTTP

1. Start the Flask service: `python serving.py` (port 8001).
2. Request a complete simulated batch: `GET /batch`.
3. Request a single reading: `GET /point?t=45`, then an invalid one: `GET /point?t=abc`.
4. Request `GET /point?t=120` several times and compare the returned `state`.

What to explain

At $t = 120\,\text{s}$ the service returns sometimes Fill and sometimes HeatUp: every request samples new phase durations. The endpoint is stateless, so successive points do not belong to the same batch.

Exercise 2: expected responses

`GET /point?t=45` (status 200)

```
{
  "Kappa": 4.653,  "Mu": 2.229,
  "P": 1.135,      "Q_in": 0.0,
  "Q_out": 0.0,    "T": 10.033,
  "Tau": 0.213,    "dTdt": -0.004,
  "pH": 6.649,
  "state": "Idle", "timestamp": 45.0
}
```

`GET /point?t=abc` (status 400)

```
{
  "error": "Invalid t parameter"
}
```

- `GET /batch` returns a JSON array of about 300–360 such objects, one per second of the batch, with an additional field `batch_id`.

Responses captured from `serving.py`; values rounded to three decimals and keys arranged in two columns for readability. Numerical values differ at every request.

Serving a single point in the code

```python
@app.route("/point")
def point():
    try:
        t = float(request.args.get("t", 0))
    except ValueError:
        return jsonify({"error": "Invalid t parameter"}), 400

    data = simulate_point(t)
    return jsonify(data)
```

- From `pasteurization/serving.py`: the input is validated, and an invalid value returns the status code 400 (Bad Request) with an error message instead of an exception.
- `simulate_point` draws a new timeline at every call, which explains the behaviour observed in Exercise 2.

Exercise 3: consume the live stream

1. With `serving.py` running, request `curl -N http://localhost:8001/stream`.
2. Observe one JSON event per second in the Server-Sent Events (SSE) format: each event is a line `data: {...}` followed by a blank line.
3. Alternatively, run `streamlit run streamlit/Main.py` and open the Pasteurization page, which consumes the same endpoint.
4. Compare the raw output of `curl` with the plots of the dashboard.

The streaming endpoint in the code

```python
@app.route("/stream")
def stream():
    df, _ = simulate_batch(BATCH_ID)

    def generate():
        for _, row in df.iterrows():
            yield f"data: {json.dumps(row.to_dict())}\n\n"
            time.sleep(STREAM_DT)

    return Response(generate(), mimetype="text/event-stream")
```

- From `pasteurization/serving.py`: a complete batch is simulated first and then replayed at one row per second as an event stream.
- Unlike `/point`, the stream is internally consistent: all events belong to one simulated batch.

Writing your own stream client

```python
import json, requests

URL = "http://localhost:8001/stream"
with requests.get(URL, stream=True, timeout=10) as r:
    for line in r.iter_lines():
        if line.startswith(b"data: "):
            event = json.loads(line[len(b"data: "):])
            print(event["timestamp"], event["state"], round(event["T"], 2))
```

- `stream=True` keeps the connection open; `iter_lines` yields each line as soon as it arrives.
- Each SSE event is a line starting with `data:  `, followed by a blank separator line; the payload after the prefix is JSON.
- Expected output: one line per second, e.g. `0.0 Idle 8.74`, `1.0 Idle 8.7`, ...

What to observe while streaming

- The states follow the fixed order Idle, Fill, HeatUp, Hold, Cool, Discharge.
- `dTdt` is close to zero during Idle and Hold, positive during HeatUp, and negative during Cool.
- `Q_in` is non-zero only during Fill; `Q_out` only during Discharge.
- Every sensor remains within the fixed axis ranges of the Streamlit dashboard.

Common pitfalls in this lab

- **Connection settings:** the IP address and port in the dashboard sidebar (default `127.0.0.1:8001`) must match the machine where `serving.py` runs.
- **Port conflicts:** check that no other process is already bound to port 8001.
- **Concurrency:** `serving.py` runs Flask with `threaded=True`; without it, an open `/stream` connection would block every other request.
- **Non-reproducible responses:** `/stream` simulates a new batch at every call, and `/point` a new timeline at every request; two clients never receive identical data.

Exercise 4: detect an injected drift (Class 18)

```python
import requests, pandas as pd
from river import linear_model, optim, preprocessing, drift

URL = "http://localhost:8001/batch"
frames = [pd.DataFrame(requests.get(URL).json()) for _ in range(30)]
stream = pd.concat(frames, ignore_index=True)
start = sum(len(f) for f in frames[:15])      # drift begins with batch 16

model = preprocessing.StandardScaler() | linear_model.LinearRegression(
    optimizer=optim.SGD(0.01))
adwin = drift.ADWIN(delta=0.002)
for i, e in stream.iterrows():
    mu = e["Mu"] + (0.4 if i >= start else 0.0)   # injected product change
    y_pred = model.predict_one({"T": e["T"]}) or 0.0
    model.learn_one({"T": e["T"]}, mu)
    adwin.update(abs(mu - y_pred))
    if adwin.drift_detected:
        print(f"drift detected at sample {i} (injected at {start})")
```

- Thirty batches are requested from `/batch` (no waiting) and processed row by row as a stream; the viscosity is shifted by 0.4 cP from batch 16 on.

Exercise 4: questions

Report the detection delay and every alarm raised before the injection. In ten test runs, the drift was detected 28–54 samples after the injection; one or two alarms during the initial learning phase (roughly the first 1,500 samples) are expected.

1. Why is `/batch` used instead of `/stream`? Estimate how long the same experiment would take through `/stream`.
2. Why do alarms appear during the initial learning phase, and why should they be ignored?
3. Reduce the shift to 0.1 cP and then to 0.05 cP: how do the detection delay and the detection rate change?
4. Repeat the experiment with only four batches and the injection after batch 2. Explain why the drift is rarely detected (hint: compare with the learning curve of the online model).
5. Replace the sudden shift by a gradual one (e.g. $+0.4$ cP reached linearly over five batches) and compare the delay.

Repository connections

Repo activity

- `pasteurization/synth_sensors.py` and `pasteurization/serving.py`: the generator and the service used throughout the lab.
- `streamlit/pages/App_Pasteurization.py`: the client that consumes `/stream` and plots every sensor live.

Papers and materials

- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine.
- WHATWG. HTML Living Standard: Server-sent events.

- Documentation: Flask (including the tutorial); Streamlit; pandas.

Take-away message

Before trusting a data service, run it and probe it:
validate its inputs, check its outputs against the expected process, and know whether its responses are reproducible.

Thank you for your attention.

Next: Class 20 — model deployment and prediction serving.